data.

The third option is slightly more complex. If you add a version number to your session keys and change that number whenever you update the stored data, you'll only ever load data that corresponds with the current version of the application. You can potentially version the classes whose objects are stored in the session and use the appropriate classes depending on the session keys associated with each request. This last idea can be a lot of work, so you'll need to decide whether it's worth the effort.

Because the session store is hash-like, you can save multiple objects in it, each with its own key.

There's no need to also disable sessions for particular actions. Because sessions are lazily loaded, simply don't reference a session in any action in which you don't need a session.

Session Storage

Rails has a number of options when it comes to storing your session data. Each has good and bad points. We'll start by listing the options and then compare them at the end.

The `session_store` attribute of `ActionController::Base` determines the session storage mechanism—set this attribute to a class that implements the storage strategy. This class must be defined in the `ActiveSupport::Cache::Store` module. You use symbols to name the session storage strategy; the symbol is converted into a CamelCase class name.

`session_store = :cookie_store`
: This is the default session storage mechanism used by Rails, starting with version 2.0. This format represents objects in their marshaled form, which allows any serializable data to be stored in sessions but is limited to 4 KB total. This is the option we used in the Depot application.

`session_store = :active_record_store`
: You can use the `activerecord-session_store` gem[104] to store your session data